

Comments and Functions in Bash (Terminal Style)

Definition:

Comments are lines ignored by the shell, used to document code, explain logic, or disable parts of a script.

Types of Comments:

# This is a comment	- Single-line comment
echo 'Hello' # Inline comment	- Inline comment
: ' ... '	- Multi-line comment using colon and quotes
<<COMMENT ... COMMENT	- Multi-line comment using here-doc

Examples:

```
# This script prints system info
echo 'System Information:'
uname -a
```

```
: '
```

Multi-line comment block.

Used to describe code sections.

```
'
```

Functions in Bash:

Functions are reusable code blocks that perform specific tasks and help avoid repetition.

Syntax:

```
function function_name { commands }
```

or

```
function_name() { commands }
```

Example 1: Basic Function

```
greet() {  
    echo 'Hello, $1!'  
}  
greet 'Alice'  
# Output: Hello, Alice!
```

Example 2: Using Local Variables

```
calculate_sum() {  
    local a=$1  
    local b=$2  
    echo 'Sum: $((a + b))'  
}  
calculate_sum 5 10  
# Output: Sum: 15
```

Example 3: Return Value

```
add() { echo $(( $1 + $2 )); }  
result=$(add 4 6)  
echo 'Result: $result'
```

Example 4: Function Calling Another

```
say_hello() { echo 'Hello!'; }  
say_goodbye() { say_hello; echo 'Goodbye!'; }  
say_goodbye
```

Example 5: Function with Condition

```
check_number() {  
    if [ $1 -gt 0 ]; then  
        echo 'Positive'  
    elif [ $1 -lt 0 ]; then
```

```

        echo 'Negative'
    else
        echo 'Zero'
    fi
}

```

Example 6: Exit Status

```

is_even() {
    if (( $1 % 2 == 0 )); then return 0; else return 1; fi
}
is_even 4 && echo 'Even' || echo 'Odd'

```

Example 7: Recursive Function

```

factorial() {
    if [ $1 -le 1 ]; then echo 1; else local prev=$(factorial $(( $1 - 1
    ))); echo $(( $1 * prev )); fi
}
result=$(factorial 5)
echo 'Factorial: $result'

```

Summary Table:

# comment	- Single-line comment
: '...'	- Multi-line comment
function_name()	- Define a function
local var=value	- Create local variable
return N	- Exit with status code
\$?	- Last command exit status
\$(command)	- Command substitution